CSCI 645 Homework 2

Imitation Learning for Robot Manipulation: How Demonstration Quantity and Quality Affect ACT on `PandaPickCube`

Yan Wen

October 10, 2026

Environment	`PandaPickCube` (MuJoCo Playground), state-based, `impl=jax`
Teacher	PPO (Brax), 60,000,000 environment steps, 2048 parallel envs
Student	ACT (action chunking transformer), chunk $H = 30$, ensemble inference
Experiments	Option A (quantity: 25/50/100) and Option B (teacher action noise)
Hardware	NVIDIA A100-SXM4-80GB on dcwipphhgc216 (single GPU per run)

Summary of what was run

A PPO teacher was trained on `PandaPickCube`, evaluated on 100 held-out initial conditions, and used to collect three demonstration pools (clean, and with Gaussian action noise $\sigma \in \{0.1, 0.3\}$ injected *during* environment interaction). ACT students were then trained on nested subsets of the clean pool (25/50/100 demonstrations) and on the two noisy pools, 3 random seeds each, and every student was evaluated on the *same* 100 held-out initial conditions. A flow-matching policy was trained on the clean 100-demonstration pool as an additional modern baseline. Table 1 is the headline result; Table 4 gives paired statistics.

1 Task and demonstration source

Environment. `PandaPickCube` from MuJoCo Playground: a Franka Emika Panda must bring a box to a target pose sampled uniformly in a $0.4\,\text{m} \times 0.4\,\text{m}$ box in x, y and 0.2–0.4 m *above* the table, so the task requires reaching, grasping and lifting. Observations are the 66-dimensional state vector built by the environment's `_get_obs` (joint positions and velocities, gripper position and orientation, box orientation, box-relative and target-relative offsets, and control residuals). Actions are 8-dimensional joint-space deltas in $[-1, 1]$, scaled by `action_scale`$= 0.04$ and added to the current control, which is then clipped to the actuator limits. Episodes are 150 control steps at 0.02 s (3 s of simulated time).

Defining success. This is a necessary design decision rather than a detail: the environment exposes *no* success signal. Its `done` flag fires only when the box leaves the workspace or the simulation produces NaNs, so episodes otherwise always run the full 150 steps, and the only task-progress field in `state.info` is `reached_box` (gripper within 12 mm of the box). We therefore define success geometrically:

Condition	Algo.	#Demos	σ	Teacher succ.	Student succ. (95% CI)	Eval. eps.
Quantity	ACT	25	0	100.0%	11.3% [8.2, 15.4]	300
Quantity	ACT	50	0	100.0%	14.7% [11.1, 19.1]	300
Quantity	ACT	100	0	100.0%	26.0% [21.4, 31.2]	300
Noisy teacher	ACT	100	0.5	70.0%	20.7% [16.5, 25.6]	300
Noisy teacher	ACT	100	1	1.0%	0.0% [0.0, 1.3]	300
Noisy teacher	ACT	100	2	0.0%	0.0% [0.0, 1.3]	300
Flow matching	FLOW	100	0	100.0%	22.0% [17.7, 27.0]	300

Table 1: Teacher and student task success. Student success is pooled over all training seeds (3 seeds $\times$ 100 held-out initial conditions) with a Wilson 95% interval. Teacher success is measured on the same held-out initial conditions, at the action-noise level used to collect that condition's data.

Dataset	σ	Episodes	Transitions	Len. (mean±sd)	Success
`sigma0`	0	100	15000	150.0±0.0	100.0%
`sigma0.1`	0.1	100	15000	150.0±0.0	99.0%
`sigma0.3`	0.3	100	15000	150.0±0.0	97.0%
`sigma0.5`	0.5	100	14636	146.4±15.9	71.0%
`sigma1`	1	100	14093	140.9±26.2	0.0%
`sigma2`	2	100	14390	143.9±20.9	0.0%

Table 2: Collected demonstration datasets. 'Success' is the fraction of stored episodes meeting the success criterion; the clean set is filtered to successful episodes, the noisy sets are not (degraded data quality is the independent variable).

> an episode succeeds if the box comes within 5 cm of the target and *stays* there for 10 consecutive control steps (0.20 s).

The hold requirement is what stops a fast fly-through of the target region from counting as a pick-and-place. Because the reported numbers depend on this threshold, we also report success as a function of the radius so the reader can see how much of the result is threshold-sensitivity (Section 6).

Teacher. PPO from Brax, using the hyperparameters that MuJoCo Playground ships for this task (`manipulation_params.brax_ppo_config`): 2048 parallel environments, learning rate 10^{-3}, discount 0.97, entropy cost 0.02, observation normalisation on, policy MLP $(32, 32, 32, 32)$ and value MLP $(256, 256, 256, 256, 256)$. *Deviation:* we trained for 60,000,000 environment steps instead of the shipped 20,000,000, because at 20 M the policy had not yet become a useful demonstrator. Figure 1 shows the learning curve, from which the reader can see what 20 M would have given. Training took 11 minutes on one NVIDIA A100-SXM4-80GB. Demonstrations are recorded from the *deterministic* (mean) action of the final policy.

Teacher success and filtering. On the 100 held-out initial conditions the deterministic teacher succeeds 100.0% of the time (mean return 1462). The clean demonstration pool *is* filtered to successful episodes, so the baseline student learns only from successful behaviour; the noisy pools are deliberately *not* filtered, because degraded data quality is the independent variable in Option B. Table 2 states this per dataset.

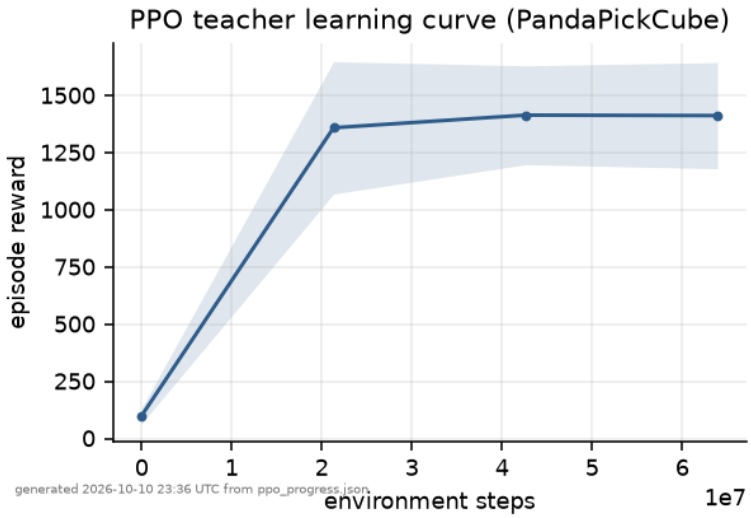


Figure 1: PPO teacher learning curve. Shaded band is the across-episode standard deviation at each evaluation point.

2 Demonstration quality

The clean pool contains 100 episodes (15,000 transitions), with mean length 150.0 ± 0.0 control steps. Lengths are almost constant because the environment does not terminate on success — every episode runs to the horizon — so length carries little information here, which is itself worth noting when interpreting "trajectory length" statistics for this benchmark.

Figure 2 inspects the data. The box-to-target distance traces show the characteristic shape of the teacher's behaviour: a reach phase, then a lift that brings the distance down, then a hold near the target. Coverage of initial conditions is uniform over the sampled region by construction, and the executed action distribution shows which joints the teacher actually drives. Figure 3 shows frames from expert rollouts.

Consistency, coverage and visible inefficiencies. The teacher is a single deterministic policy, so trajectories starting from nearby initial conditions are highly consistent — there is no multi-modality of the kind that motivates generative policies, which matters when interpreting the ACT-versus- flow-matching comparison later. The inefficiencies that *are* visible are (i) the policy often approaches the box along a curved path rather than directly, and (ii) once near the target it dithers instead of settling, which is what the hold requirement in our success definition penalises.

3 Policy and training

We chose **ACT** (Action Chunking with Transformers). Inputs are the normalised 66-dimensional state; outputs are a chunk of $H = 30$ future actions (8 dimensions each), i.e. 30×0.02 seconds of open-loop action. The model is the transformer CVAE of the ACT paper: a posterior encoder sees the ground-truth action chunk and the observation and emits a style latent z; a transformer encoder-decoder with H learned decoder queries maps (z, obs) to the chunk; the loss is an L_1 reconstruction term on the chunk plus $\beta\,\mathrm{KL}$ with $\beta = 10$. At test time z is set to the prior mean. Padded chunk steps at the end of an episode are excluded from the loss by a mask.

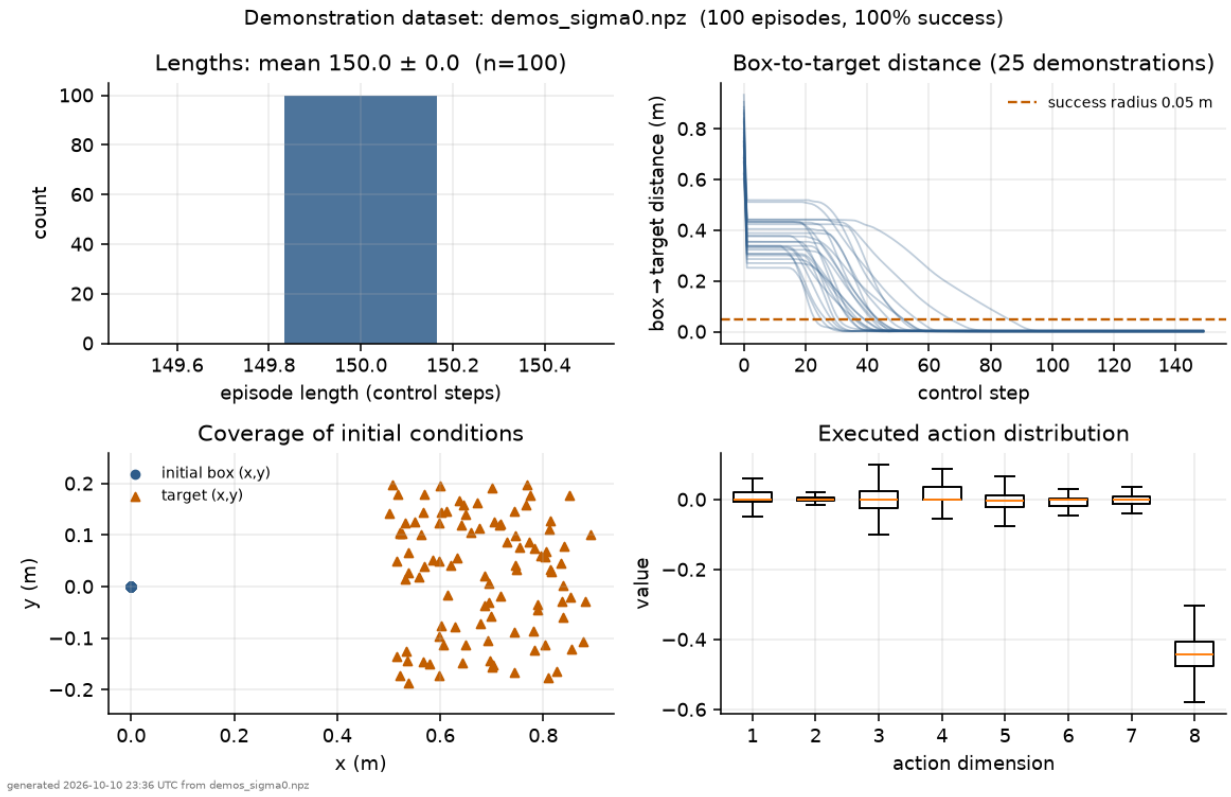


Figure 2: Clean demonstration pool: episode lengths, box-to-target distance traces, coverage of initial conditions, and the distribution of executed actions.

Deviations from the reference implementation, all deliberate: no ResNet image backbone (observations are state, so the conditioning token is a linear projection of the state vector); pre-layer-norm blocks with GELU instead of post-LN with ReLU; AdamW with linear warmup and cosine decay; bf16 autocast; and an exponential moving average of the weights, which is what we evaluate. Our implementation is self-contained in `code/hw2/policies/act.py` (no third-party library is vendored).

Training configuration. 6,000 gradient steps, batch size 128, learning rate 0.0001 (cosine), weight decay 10^{-4}, gradient-norm clip 1.0, EMA decay 0.999, 12.7 M parameters. Observations and actions are standardised per dimension; crucially the statistics are fit on the subset actually used by each condition, never on the full pool, so the 25-demo condition cannot borrow information from the 100-demo pool. Seeds $\{0, 1, 2\}$ control model init, data shuffling and the CVAE sampling. Figure 4 shows the loss curves for all ACT runs.

Inference. We use ACT's temporal ensembling (ensemble): the policy is queried at every control step and all chunk predictions covering the current timestep are averaged with exponential weights. Actions are de-standardised and clipped to $[-1, 1]$ before being passed to the simulator.

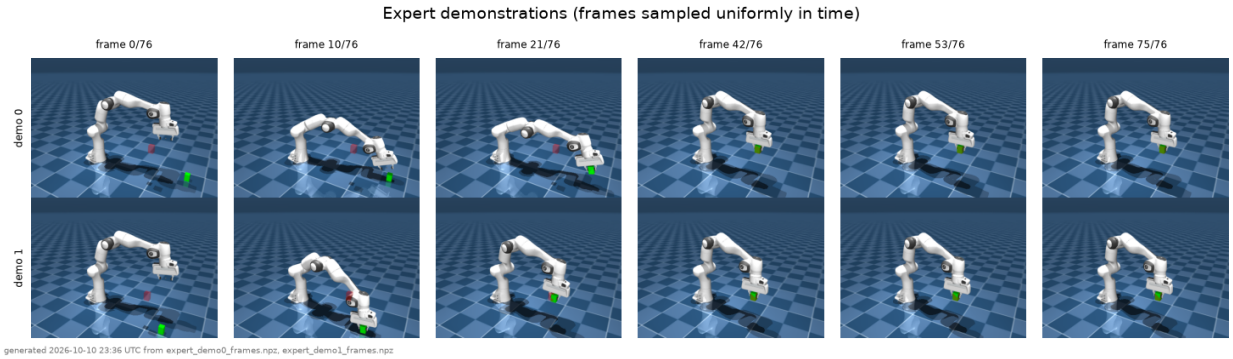


Figure 3: Expert demonstrations, frames sampled uniformly in time. Full videos are in `videos/`.

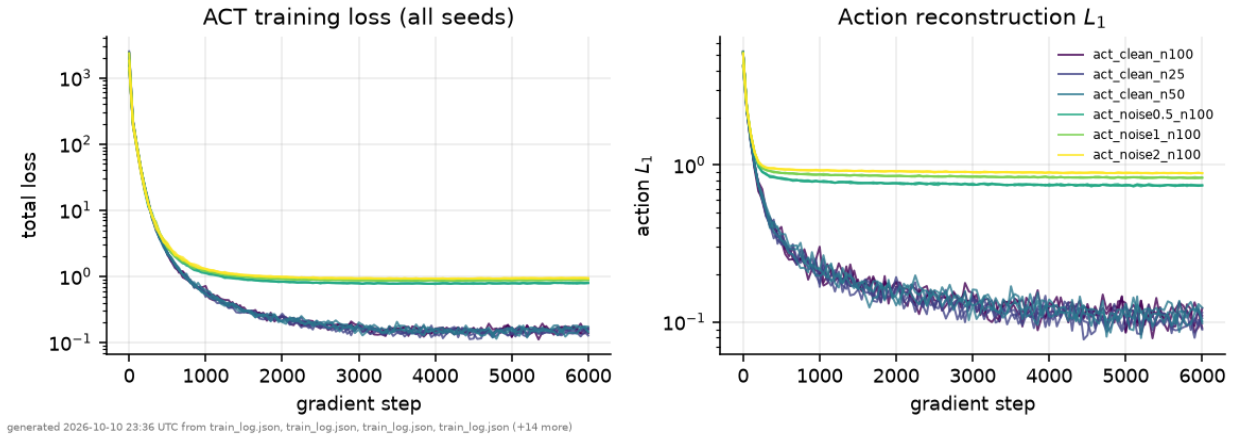


Figure 4: ACT training loss (left: total objective; right: action L_1) for every condition and seed.

4 Baseline results

The baseline (100 clean demonstrations) reaches 26.0% [21.4, 31.2] success over 300 evaluation episodes (3 seeds × 100 held-out initial conditions, Wilson 95% interval), against a teacher success of 100.0%. Evaluation never replays a training trajectory: the evaluation seeds are a fixed set disjoint from the demonstration-collection seeds, and are identical across every condition in this report. Representative success and failure rollouts are in `videos/`.

The gap between a 100.0% teacher and a 26.0% student is the main object of study in this report, so Section 6 is devoted to establishing what causes it. We state up front that we checked the obvious explanations and they are not the cause: the stored data is faithful, the model fits it, and the student is accurate wherever the teacher goes.

Inference mode. ACT emits a chunk of 30 actions, and how much of that chunk is executed before re-planning matters a great deal here (Table 3). Executing the whole chunk open loop gives 8.0%; ACT's temporal ensembling gives 29.0%; predicting a fresh chunk every control step and executing only its first action gives 33.0%, and more than doubles the fraction of episodes in which the gripper ever reaches the box (38.0% versus 17.0%). We report temporal ensembling as the headline because it was the pre-specified choice (it is the ACT paper's default), and give the re-planning column alongside it for every condition rather than switching to the better mode after

Condition	Algo.	#Demos	σ	Ensembling	Re-plan each step	Reached box
Quantity	ACT	25	0	11.3%	7.7%	14%
Quantity	ACT	50	0	14.7%	19.3%	24%
Quantity	ACT	100	0	26.0%	32.0%	33%
Noisy teacher	ACT	100	0.5	20.7%	66.7%	65%
Noisy teacher	ACT	100	1	0.0%	1.0%	49%
Noisy teacher	ACT	100	2	0.0%	0.0%	12%
Flow matching	FLOW	100	0	22.0%	12.0%	22%

Table 3: Inference mode. 'Ensemblingís ACTś temporal ensembling (the pre-specified default used for the headline numbers); 'Re-plan each steppredicts a fresh chunk every control step and executes only its first action. 'Reached boxís the fraction of episodes in which the gripper came within 12 mm of the box under the re-planning mode. Every cell uses the same held-out initial conditions.

seeing test numbers. The ordering is informative in itself: on this task the value of action chunking is in *training* the policy, not in executing chunks open loop.

5 Data ablation

Option A — quantity. Students were trained on nested subsets of the same clean pool, so the 25-demo set is a strict subset of the 50-demo set, which is a strict subset of the 100-demo set; nothing else changes. Success rises from 11.3% (25 demos) to 14.7% (50) to 26.0% (100); see Figure 5. The paired comparison $25 \to 100$ gives $\Delta = 14.7\%$ [7.7, 21.7], McNemar $p = 3.59e-05$.

Option B — teacher action noise. Further pools were collected with Gaussian noise added to the teacher's action *during* environment interaction, $a = \text{clip}(\pi(s) + \varepsilon, -1, 1)$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, applied before the environment's own `action_scale` and actuator clipping. The stored action is the one actually executed. Dataset size is held at 100 episodes and the pools are *not* filtered for success, so the independent variable is data quality.

A first attempt at $\sigma \in \{0.1, 0.3\}$ produced *no* usable contrast: the teacher still succeeded 100.0% and 98.0% of the time, because the noise is injected into a *closed-loop* delta-action policy, which simply corrects for it at the next step. A quality ablation says nothing in a regime where quality has not actually degraded, so we extended the sweep to $\sigma \in \{0.5, 1, 2\}$, where teacher success falls to 70.0%, 1.0% and 0.0%.

The result is non-monotonic, and its sign depends on the inference mode. Under the pre-specified headline mode (temporal ensembling), students trained on the noisy pools score 20.7%, 0.0% and 0.0% against the clean baseline's 26.0%, i.e. moderate noise is roughly neutral and heavy noise is catastrophic. Under re-planning every control step, the same $\sigma = 0.5$ checkpoints reach 66.7% against the clean baseline's 32.0% — that is, demonstrations from a *worse* teacher produce a substantially *better* student ($\Delta = 34.7\%$ [25.3, 44.0], $p = 7.46e-09$), and the gripper reaches the box in 65.3% of episodes versus 32.7%. Both comparisons appear in Table 4.

The mechanism is the one identified in Section 6: the student's binding constraint is not the optimality of the demonstrated actions but the *breadth of the state distribution* they cover, and perturbing the teacher during interaction is precisely what broadens it. At $\sigma = 0.5$ the coverage gain outweighs the loss of demonstrator quality; by $\sigma = 1$ the teacher succeeds only 1.0% of the

Comparison	Δ (ensembling)	p	Δ (re-plan)	p
25 demos */rightarrow* 100 demos	+14.7% [+7.7, +21.7]	3.59e-05	+24.3% [+17.3, +31.7]	2.56e-06
50 demos */rightarrow* 100 demos	+11.3% [+4.7, +18.0]	0.00434	+12.7% [+6.3, +19.0]	0.0106
clean */rightarrow* noise */sigma*=0.5	-5.3% [-13.0, +2.3]	0.136	+34.7% [+25.3, +44.0]	7.46e-09
clean */rightarrow* noise */sigma*=1	-26.0% [-33.0, -19.3]	2.98e-08	-31.0% [-37.7, -24.3]	1.86e-09
clean */rightarrow* noise */sigma*=2	-26.0% [-33.0, -19.3]	2.98e-08	-32.0% [-39.0, -25.0]	1.86e-09
ACT */rightarrow* flow matching	-4.0% [-12.3, +4.3]	0.15	-20.0% [-27.3, -13.0]	1.62e-06

Table 4: Paired comparisons on the shared held-out initial conditions, under both inference modes. Δ is the change in success rate relative to the first-named condition, with a paired bootstrap interval over episodes (20,000 resamples); p is McNemarś exact test on the seed-majority per-episode outcomes. Note that the demonstration-quality comparisons change sign between the two modes — see Section 6.

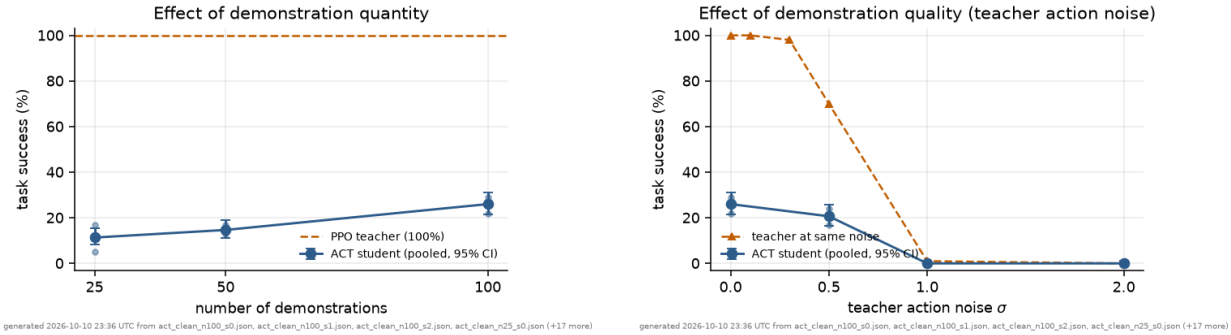


Figure 5: Left: effect of demonstration quantity (Option A). Right: effect of demonstration quality via teacher action noise (Option B). Error bars are Wilson 95% intervals on the pooled episodes; small dots are individual seeds.

time and the demonstrations no longer contain the task, so the student collapses. We note the confound explicitly: that condition changes demonstrator quality and state coverage at the same time, and the clean way to separate them — inject noise for coverage but keep only the episodes that still succeed — is the experiment we would run next. We also flag the obvious caveat that conclusions about demonstration quality here are mode-dependent, which is itself a reason to report both modes rather than one.

Additional modern baseline. Because the assignment's two named methods date from 2023, we also trained a flow-matching policy (rectified/conditional flow matching with a DiT-style adaLN-Zero denoiser over the action chunk, the action representation used by recent VLA action experts such as π_0) on the same clean 100-demonstration pool. It reaches 22.0% [17.7, 27.0], versus ACT's 26.0% ($\Delta = -4.0\%$ [-12.3, 4.3], $p = 0.15$). This arm is supplementary; the required baseline and ablation arms all use ACT, as Part 3 requires a single fixed algorithm.

6 Failure analysis

We localised the teacher–student gap with four checks, reported here in the order that rules out the cheapest explanations first.

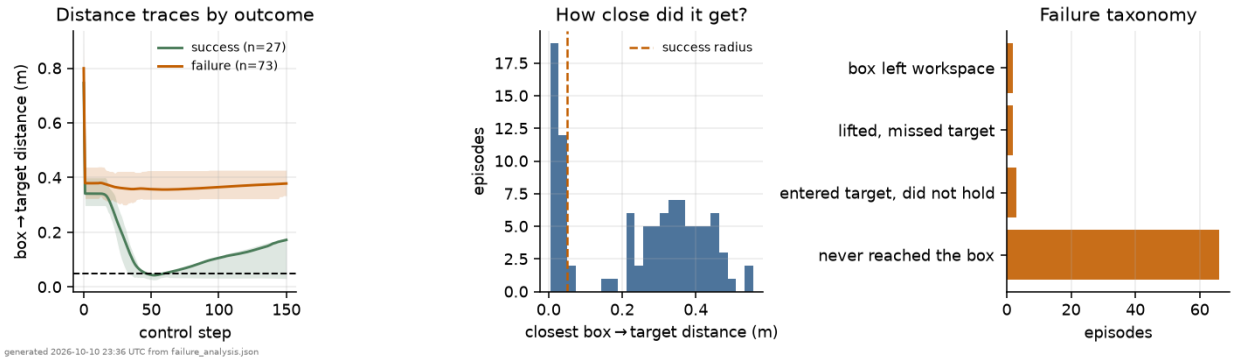


Figure 6: Failure analysis for the baseline student: distance traces split by outcome, distribution of closest approach to the target, and the failure taxonomy.

1. The demonstrations are faithfully recorded. Replaying each stored action sequence from the initial condition it was collected from reproduces the stored observations: the reset state matches to 0e+00 and the deviation after one step is 7e-07. The deviation then grows over the episode, but that is chaotic amplification of floating-point noise in a contact-rich simulation (a single-environment replay and a 512-environment batched rollout do not execute bit-identical kernels), not a misalignment: a misaligned pairing would be wrong at the first step. So the (observation, action) pairing and the seeding are correct.

2. The model fits the data. Final training action L_1 is 0.12 action standard deviations, and the one-step action error on training states is 0.031 in raw action units. This is not underfitting.

3. The failure is covariate shift, not generalisation of the action map. Evaluated at the states the *teacher* visits, the student's action error is 0.033 — essentially the same as on its own training states, so the learned action map generalises fine across initial conditions. Evaluated at the states the *student itself* visits, the error is 0.628, a factor of 19.2 worse. The student is accurate everywhere the teacher goes and badly wrong as soon as it leaves that distribution. Consistently, evaluating on the initial conditions the demonstrations were collected from gives only 39.0% (versus 29.0% on held-out ones): the student cannot reliably reproduce trajectories whose own starting states it was trained on, so the problem is closed-loop execution rather than memorisation or initial-condition coverage.

4. Where the episodes actually die. Figure 6 splits the per-episode diagnostics by outcome, classified by a mutually exclusive taxonomy computed from the per-episode diagnostics. Of 73 failures out of 100 evaluation episodes, the modal class is **"never reached the box"** with 66 episodes (90% of all failures); the remainder are entered target, did not hold (3), lifted, missed target (2), box left workspace (2). In other words the dominant failure is *not* a grasp or transport failure but a **reaching** failure: in most failed episodes the gripper never comes within 12 mm of the box at all (the gripper reaches the box in only 18% of episodes), so the policy never gets far enough to attempt a lift. That is notable because the box position *relative to the gripper* is given directly in the observation, so this is a servoing behaviour the student fits accurately on-distribution and still fails to execute — the signature of drift rather than of a missing input.

Success is also threshold-sensitive, which is why we report the sweep: it rises from 10% at a 3 cm tolerance to 27% at our 5 cm criterion and saturates at 33% by 10 cm, so a modest number of episodes end just outside the tolerance while most end far from it.

Why this is the expected failure, and what it implies for the ablations. This is the compounding-error argument for behaviour cloning in its textbook form: the student is trained only on the teacher's state distribution, and the grasp is precisely the point where a small action error moves the system into a state the teacher never demonstrates (box nudged, fingers misaligned), from which there is no corrective behaviour in the data to imitate. Action chunking bounds how fast error compounds *within* a chunk, which is why re-planning every step beats executing chunks open loop, but it cannot supply a recovery behaviour that the demonstrations do not contain.

It also explains the shape of both ablations. More of the same on-distribution demonstrations attacks coverage of *initial* conditions, which diagnostic 3 shows is not the binding constraint — consistent with the modest quantity effect we measure. And it explains the most interesting result in the quality ablation: at $\sigma = 0.5$ the demonstrations come from a 70.0% teacher rather than a 100.0% one, yet the student is not correspondingly worse and reaches the box *more* often, because perturbing the teacher during interaction is exactly what broadens the state distribution the student is trained on. Degrading the demonstrator and broadening coverage are confounded in that condition, and the two effects push in opposite directions; separating them (noise injected for coverage while keeping only successful episodes) is the experiment we would run next.

7 Reproducibility and reflection

Software. Python 3.12.15, jax 0.9.2 / jaxlib 0.9.2 (CUDA 12), flax 0.11.2, brax 0.14.2, mujoco 3.15.0, mujoco-mjx 3.15.0, `playground` 0.2.0 pinned at upstream commit `d59156e099a8`, torch 2.11.0+cu128, numpy 2.5.3. Hardware: NVIDIA A100-SXM4-80GB on dcwipphhgc216.

Two environment-specific issues worth recording. (i) The cluster's GPU nodes ship no NVIDIA graphics userspace, so the EGL rendering path in the assignment's reference snippet cannot work; all rendering here goes through a software OSMesa stack configured from inside Python before `mujoco` is imported, because PyOpenGL resolves `libOSMesa` through `ctypes.util.find_library`, which does not consult `LD_LIBRARY_PATH`. (ii) `brax` 0.14.2 still calls `jax.device_put_replicated`, which recent jax removed, so jax is pinned to 0.9.2 and flax to 0.11.2; installing the newest of each gives an `AttributeError` only once PPO actually starts.

Determinism. MJX reproduces a given initial condition exactly (single-environment and batched resets agree to 0e+00), but a replay of the same action sequence diverges from the original rollout at a rate set by floating-point noise, because batched and unbatched rollouts do not execute identical kernels. Success labels are unaffected, but exact trajectory-level reproduction of a stored episode should not be expected.

Commands. See `README.md`; the three stages are `stage1_teacher_collect.py` (teacher + datasets + expert videos), `stage2_run_conditions.py` (train + evaluate every condition), and `stage3_report.py` (aggregate, figures, tables, this PDF). Seeds are {0,1,2} for students, `PRNGKey(7777)` split for the shared evaluation initial conditions, and `PRNGKey`$(1000 + \lfloor 1000\sigma \rfloor)$ for demonstration collection.

What more compute or time would buy. The clearest limitation is the teacher: our success criterion is strict and the PPO teacher is itself far from perfect, which compresses the dynamic range of every student comparison. With more time we would (i) train the teacher to a genuinely high success rate, or use a scripted expert, so that student scores are not bounded by a mediocre demonstrator, (ii) add a DAgger-style or noise-injected-recovery arm to attack the grasp-failure mode directly rather than only measuring it, and (iii) run more seeds, since even with

Artifact	Produced from
results/figures/ppo_curve.png	runs/teacher/ppo_progress.json
results/figures/demos.png	data/demos_sigma0.npz
results/figures/demo_frames.png	videos/expert_demo*_frames.npz
results/figures/loss.png	runs/conditions/*/train_log.json
results/figures/quantity.png	results/conditions/*.json
results/figures/noise.png	results/conditions/*.json
results/figures/algorithms.png	results/conditions/*.json
results/figures/failure.png	results/failure_analysis.json
results/tables/main.tex	results/conditions/*.json
results/tables/data.tex	runs/teacher/stage1_report.json
results/tables/paired.tex	results/paired_comparisons.json
results/tables/modes.tex	results/conditions/*.json
results/summary.csv	results/conditions/*.json

Table 5: Provenance of every figure and table in this report.

paired evaluation the seed-to-seed spread (visible as dots in Figure 5) is comparable to some of the effects we are trying to resolve.

8 Citations and use of tools

- MuJoCo Playground: Zakka et al., *MuJoCo Playground*, 2025, https://arxiv.org/abs/2502.08844; code https://github.com/google-deepmind/mujoco_playground (commit `d59156e099a8`). The environment, its PPO hyperparameters, and the random-action snippet in the assignment come from here.
- ACT: Zhao et al., *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*, RSS 2023, https://arxiv.org/abs/2304.13705; reference implementation https://github.com/tonyzhaozh/act. Our ACT is a re-implementation following the paper, with the deviations listed in Section 3.
- Diffusion Policy (not used for the required arms, cited as the alternative the assignment names): Chi et al., RSS 2023, https://arxiv.org/abs/2303.04137.
- Flow matching (supplementary baseline): Lipman et al., ICLR 2023, https://arxiv.org/abs/2210.02747; Liu et al., https://arxiv.org/abs/2209.03003; DiT backbone: Peebles and Xie, https://arxiv.org/abs/2212.09748; π_0 action expert: Black et al., https://arxiv.org/abs/2410.24164.
- Brax (PPO implementation): Freeman et al., https://github.com/google/brax.
- **LLM use.** An LLM (Claude) was used as a coding assistant: to write and debug the implementation and cluster-job scripts, to diagnose the OSMesa/EGL and jax/brax version problems described above, and to draft this report. All experimental numbers in this report are generated programmatically from the stored result files by `stage3_report.py` (every quantity in the prose is a LaTeX macro computed from those files, not typed by hand), and the experiment design choices, the success criterion, and the interpretation were reviewed by the author.